

 Cloudformation

AWS CloudFormation: Do Zero ao Avançado

Curso completo com 4 módulos

Templates • VPC • EC2 • RDS • S3 • Nested Stacks

CloudTrilhas — cloudtrilhas.com.br

Por Ricardo Simines Scopim

 CloudFormation — Módulo 01

Fundamentos: Template Structure e Intrinsic Functions

CloudFormation é o serviço nativo da AWS para provisionar infraestrutura como código usando templates YAML/JSON declarativos.

1. Estrutura do Template

Template Base **ESTRUTURA**

Todo template CloudFormation segue uma estrutura com seções opcionais e obrigatórias. Apenas **Resources** é obrigatório.

```
AWSTemplateFormatVersion: "2010-09-09"

Description: "Template de exemplo – infraestrutura básica"

Parameters:

  Environment:

    Type: String

    Default: dev

    AllowedValues:

      - dev

      - staging

      - prod

    Description: "Ambiente de deploy"

  InstanceType:

    Type: String

    Default: t3.micro

    AllowedValues:

      - t3.micro

      - t3.small

      - t3.medium

Mappings:

  RegionAMI:

    us-east-1:

      HVM64: ami-0c02fb55956c7d316

    sa-east-1:

      HVM64: ami-0b6c2d49148000cd5

Conditions:

  IsProd: !Equals [!Ref Environment, prod]

Resources:

  # Recursos AWS aqui

Outputs:

  StackEnvironment:

    Description: "Ambiente do stack"

    Value: !Ref Environment
```

```
Export:
  Name: !Sub "${AWS::StackName}-Environment"
```

2. Parameters — Tipos e Validação

Parameters ENTRADA

Parameters permitem customizar stacks no momento do deploy sem alterar o template.

```
Parameters:
  VpcCidr:
    Type: String
    Default: "10.0.0.0/16"
    AllowedPattern: '(\d{1,3})\.\d{1,3}\.\d{1,3}\.\d{1,3}/(\d{1,2})'
    ConstraintDescription: "Deve ser um CIDR válido (ex: 10.0.0.0/16)"

  KeyPairName:
    Type: AWS::EC2::KeyPair::KeyName
    Description: "Par de chaves EC2 existente"

  SubnetIds:
    Type: List<AWS::EC2::Subnet::Id>
    Description: "Lista de subnets para deploy"

  DBPassword:
    Type: String
    NoEcho: true
    MinLength: 8
    MaxLength: 41
    Description: "Senha do banco de dados (não exibida)"

  LatestAmiId:
    Type: AWS::SSM::Parameter::Value<AWS::EC2::Image::Id>
    Default: /aws/service/ami-amazon-linux-latest/amzn2-ami-hvm-x86_64-gp2
    Description: "AMI mais recente via SSM Parameter Store"
```

3. Intrinsic Functions

Funções Intrínsecas

REFERÊNCIAS

Funções intrínsecas permitem referenciar recursos, concatenar strings e aplicar lógica condicional.

```

# !Ref – referencia parâmetro ou recurso (retorna ID)
SecurityGroup:
  Type: AWS::EC2::SecurityGroup
  Properties:
    VpcId: !Ref MyVPC

# !Sub – substituição de variáveis em strings
Tags:
  - Key: Name
    Value: !Sub "${AWS::StackName}-web-server"

# !GetAtt – obtém atributo de um recurso
Outputs:
  VpcId:
    Value: !GetAtt MyVPC.VpcId
  PublicIP:
    Value: !GetAtt EC2Instance.PublicIp

# !Join – concatena lista com delimitador
UserData:
  Fn::Base64:
    !Join
      - "\n"
      - - "#!/bin/bash"
        - "yum update -y"
        - "yum install -y httpd"

# !Select – seleciona item de uma lista por índice
AvailabilityZone: !Select [0, !GetAZs ""]

# !If – valor condicional baseado em Condition
InstanceType: !If [IsProd, t3.large, t3.micro]

# Fn::ImportValue – importa output de outro stack
SubnetId:
  Fn::ImportValue: !Sub "${NetworkStackName}-PublicSubnet1"

```

4. AWS CLI — Comandos Essenciais

`aws cloudformation` CLI

Comandos para gerenciar stacks via terminal.


```
# Validar template antes do deploy
aws cloudformation validate-template \
    --template-body file:///template.yaml

# Criar stack
aws cloudformation create-stack \
    --stack-name minha-stack \
    --template-body file:///template.yaml \
    --parameters ParameterKey=Environment,ParameterValue=dev \
        ParameterKey=InstanceType,ParameterValue=t3.micro \
    --capabilities CAPABILITY_NAMED_IAM \
    --tags Key=Project,Value=CloudTrilhas

# Atualizar stack
aws cloudformation update-stack \
    --stack-name minha-stack \
    --template-body file:///template.yaml \
    --parameters ParameterKey=Environment,ParameterValue=prod

# Criar Change Set (preview de mudanças)
aws cloudformation create-change-set \
    --stack-name minha-stack \
    --change-set-name update-v2 \
    --template-body file:///template.yaml

# Descrever Change Set
aws cloudformation describe-change-set \
    --stack-name minha-stack \
    --change-set-name update-v2

# Executar Change Set
aws cloudformation execute-change-set \
    --stack-name minha-stack \
    --change-set-name update-v2

# Descrever stack e outputs
aws cloudformation describe-stacks \
    --stack-name minha-stack \
    --query "Stacks[0].Outputs"

# Listar eventos (troubleshooting)
```

```
aws cloudformation describe-stack-events \  
  --stack-name minha-stack \  
  --query "StackEvents[?ResourceStatus=='CREATE_FAILED']"\  
  
# Deletar stack  
aws cloudformation delete-stack \  
  --stack-name minha-stack\  
  
# Aguardar conclusão  
aws cloudformation wait stack-create-complete \  
  --stack-name minha-stack
```

 CloudFormation — Módulo 02

Computação: VPC + EC2 com SecurityGroups e UserData

Provisione uma VPC completa com subnets, internet gateway, security groups e instâncias EC2 com bootstrap via UserData.

1. VPC e Networking

VPC + Subnets

REDE

Criação de VPC com subnets públicas e privadas, Internet Gateway e Route Tables.

```

Resources:

  VPC:
    Type: AWS::EC2::VPC
    Properties:
      CidrBlock: !Ref VpcCidr
      EnableDnsSupport: true
      EnableDnsHostnames: true
      Tags:
        - Key: Name
          Value: !Sub "${AWS::StackName}-vpc"

  InternetGateway:
    Type: AWS::EC2::InternetGateway
    Properties:
      Tags:
        - Key: Name
          Value: !Sub "${AWS::StackName}-igw"

  AttachGateway:
    Type: AWS::EC2::VPCGatewayAttachment
    Properties:
      VpcId: !Ref VPC
      InternetGatewayId: !Ref InternetGateway

  PublicSubnet1:
    Type: AWS::EC2::Subnet
    Properties:
      VpcId: !Ref VPC
      CidrBlock: "10.0.1.0/24"
      AvailabilityZone: !Select [0, !GetAZs ""]
      MapPublicIpOnLaunch: true
      Tags:
        - Key: Name
          Value: !Sub "${AWS::StackName}-public-1"

  PrivateSubnet1:
    Type: AWS::EC2::Subnet
    Properties:
      VpcId: !Ref VPC
      CidrBlock: "10.0.10.0/24"
      AvailabilityZone: !Select [0, !GetAZs ""]

```

```
Tags:
  - Key: Name
    Value: !Sub "${AWS::StackName}-private-1"

PublicRouteTable:
  Type: AWS::EC2::RouteTable
  Properties:
    VpcId: !Ref VPC

PublicRoute:
  Type: AWS::EC2::Route
  DependsOn: AttachGateway
  Properties:
    RouteTableId: !Ref PublicRouteTable
    DestinationCidrBlock: "0.0.0.0/0"
    GatewayId: !Ref InternetGateway

PublicSubnet1RouteAssoc:
  Type: AWS::EC2::SubnetRouteTableAssociation
  Properties:
    SubnetId: !Ref PublicSubnet1
    RouteTableId: !Ref PublicRouteTable
```

2. Security Groups

SecurityGroup **SEGURANÇA**

Security Groups atuam como firewalls virtuais controlando tráfego de entrada e saída.

```

WebServerSG:
  Type: AWS::EC2::SecurityGroup
  Properties:
    GroupDescription: "Security Group para Web Server"
    VpcId: !Ref VPC
    SecurityGroupIngress:
      - IpProtocol: tcp
        FromPort: 80
        ToPort: 80
        CidrIp: "0.0.0.0/0"
        Description: "HTTP público"
      - IpProtocol: tcp
        FromPort: 443
        ToPort: 443
        CidrIp: "0.0.0.0/0"
        Description: "HTTPS público"
      - IpProtocol: tcp
        FromPort: 22
        ToPort: 22
        CidrIp: !Ref SSHCidr
        Description: "SSH restrito"
    SecurityGroupEgress:
      - IpProtocol: "-1"
        CidrIp: "0.0.0.0/0"
        Description: "Saída irrestrita"
  Tags:
    - Key: Name
      Value: !Sub "${AWS::StackName}-web-sg"

```

3. EC2 Instance com UserData e CreationPolicy

EC2 + UserData **COMPUTAÇÃO**

UserData executa scripts no boot da instância. CreationPolicy aguarda sinal de sucesso antes de marcar como CREATE_COMPLETE.


```

WebServer:
  Type: AWS::EC2::Instance
  DependsOn: AttachGateway
  CreationPolicy:
    ResourceSignal:
      Count: 1
      Timeout: PT10M
  Properties:
    ImageId: !FindInMap [RegionAMI, !Ref "AWS::Region", HVM64]
    InstanceType: !Ref InstanceType
    KeyName: !Ref KeyPairName
    SubnetId: !Ref PublicSubnet1
    SecurityGroupIds:
      - !Ref WebServerSG
    Tags:
      - Key: Name
        Value: !Sub "${AWS::StackName}-web-server"
    UserData:
      Fn::Base64: !Sub |
        #!/bin/bash -xe
        yum update -y
        yum install -y httpd aws-cfn-bootstrap

        # Configurar Apache
        systemctl start httpd
        systemctl enable httpd
        echo "<h1>Stack: ${AWS::StackName}</h1>" > /var/www/html/index.html
        echo "<p>Região: ${AWS::Region}</p>" >> /var/www/html/index.html

        # Sinalizar sucesso ao CloudFormation
        /opt/aws/bin/cfn-signal -e $? \
          --stack ${AWS::StackName} \
          --resource WebServer \
          --region ${AWS::Region}

Outputs:
  WebServerPublicIP:
    Description: "IP público do Web Server"
    Value: !GetAtt WebServer.PublicIp
  WebServerURL:

```

Description: "URL do Web Server"

Value: !Sub "http://\${WebServer.PublicDnsName}"

 CloudFormation — Módulo 03

Banco de Dados: VPC + EC2 + RDS com Conditions e Mappings

Adicione um banco de dados RDS à infraestrutura com subnets dedicadas, conditions para multi-ambiente e mappings regionais.

1. Conditions e Mappings

Conditions LÓGICA

Conditions controlam a criação condicional de recursos baseado em parâmetros.

```
Conditions:
  IsProd:
    !Equals [!Ref Environment, prod]
  IsNotProd:
    !Not [!Equals [!Ref Environment, prod]]
  CreateReadReplica:
    !And
      - !Equals [!Ref Environment, prod]
      - !Equals [!Ref EnableReplica, "true"]
  IsUsEast1:
    !Equals [!Ref "AWS::Region", us-east-1]

# Uso em Resources

Resources:
  ReadReplica:
    Type: AWS::RDS::DBInstance
    Condition: CreateReadReplica
    Properties:
      SourceDBInstanceIdentifier: !Ref Database
      DBInstanceClass: db.t3.medium
```

Mappings **LOOKUP**

Mappings são tabelas de lookup estáticas para valores por região ou ambiente.

```
Mappings:
  EnvironmentConfig:
    dev:
      InstanceType: db.t3.micro
      MultiAZ: "false"
      BackupRetention: 1
      AllocatedStorage: 20
    staging:
      InstanceType: db.t3.small
      MultiAZ: "false"
      BackupRetention: 3
      AllocatedStorage: 50
    prod:
      InstanceType: db.r5.large
      MultiAZ: "true"
      BackupRetention: 7
      AllocatedStorage: 100

# Uso com FindInMap
Resources:
  Database:
    Properties:
      DBInstanceClass: !FindInMap
        - EnvironmentConfig
        - !Ref Environment
        - InstanceType
      MultiAZ: !FindInMap
        - EnvironmentConfig
        - !Ref Environment
        - MultiAZ
```

2. DBSubnetGroup e RDS Instance

RDS **BANCO DE DADOS**

RDS requer um DBSubnetGroup com pelo menos 2 subnets em AZs diferentes para alta disponibilidade.

```

PrivateSubnet2:
  Type: AWS::EC2::Subnet
  Properties:
    VpcId: !Ref VPC
    CidrBlock: "10.0.11.0/24"
    AvailabilityZone: !Select [1, !GetAZs ""]
    Tags:
      - Key: Name
        Value: !Sub "${AWS::StackName}-private-2"

DatabaseSubnetGroup:
  Type: AWS::RDS::DBSubnetGroup
  Properties:
    DBSubnetGroupDescription: "Subnets para RDS"
    SubnetIds:
      - !Ref PrivateSubnet1
      - !Ref PrivateSubnet2
    Tags:
      - Key: Name
        Value: !Sub "${AWS::StackName}-db-subnet-group"

DatabaseSG:
  Type: AWS::EC2::SecurityGroup
  Properties:
    GroupDescription: "Security Group para RDS"
    VpcId: !Ref VPC
    SecurityGroupIngress:
      - IpProtocol: tcp
        FromPort: 3306
        ToPort: 3306
        SourceSecurityGroupId: !Ref WebServerSG
    Description: "MySQL apenas do Web Server"

Database:
  Type: AWS::RDS::DBInstance
  DeletionPolicy: Snapshot
  UpdateReplacePolicy: Snapshot
  Properties:
    DBInstanceIdentifier: !Sub "${AWS::StackName}-db"
    Engine: mysql
    EngineVersion: "8.0"

```

```

DBInstanceClass: !FindInMap [EnvironmentConfig, !Ref Environment, InstanceType]
AllocatedStorage: !FindInMap [EnvironmentConfig, !Ref Environment, AllocatedStorage]
StorageType: gp3
MasterUsername: admin
MasterUserPassword: !Ref DBPassword
DBSubnetGroupName: !Ref DatabaseSubnetGroup
VPCSecurityGroups:
  - !Ref DatabaseSG
MultiAZ: !FindInMap [EnvironmentConfig, !Ref Environment, MultiAZ]
BackupRetentionPeriod: !FindInMap [EnvironmentConfig, !Ref Environment, BackupRetentionPeriod]
DeletionProtection: !If [IsProd, true, false]
StorageEncrypted: true
Tags:
  - Key: Name
    Value: !Sub "${AWS::StackName}-database"
  - Key: Environment
    Value: !Ref Environment

Outputs:

DatabaseEndpoint:
  Description: "Endpoint do RDS"
  Value: !GetAtt Database.Endpoint.Address
  Export:
    Name: !Sub "${AWS::StackName}-DBEndpoint"

DatabasePort:
  Description: "Porta do RDS"
  Value: !GetAtt Database.Endpoint.Port

```

 CloudFormation — Módulo 04

Storage e Arquitetura: S3, Nested Stacks e Cross-Stack References

Complete a arquitetura com S3 Buckets, políticas de acesso, lifecycle rules, nested stacks e referências entre stacks.

1. S3 Bucket com Policy e Lifecycle

S3 Bucket STORAGE

S3 Buckets com versionamento, criptografia, lifecycle rules e bucket policy para controle de acesso.

```

AppBucket:
  Type: AWS::S3::Bucket
  DeletionPolicy: Retain
  Properties:
    BucketName: !Sub "${AWS::StackName}-app-${AWS::AccountId}"
    VersioningConfiguration:
      Status: Enabled
    BucketEncryption:
      ServerSideEncryptionConfiguration:
        - ServerSideEncryptionByDefault:
            SSEAlgorithm: AES256
    PublicAccessBlockConfiguration:
      BlockPublicAcls: true
      BlockPublicPolicy: true
      IgnorePublicAcls: true
      RestrictPublicBuckets: true
    LifecycleConfiguration:
      Rules:
        - Id: TransitionToIA
          Status: Enabled
          Transitions:
            - StorageClass: STANDARD_IA
              TransitionInDays: 30
            - StorageClass: GLACIER
              TransitionInDays: 90
          NoncurrentVersionExpiration:
            NoncurrentDays: 60
        - Id: CleanupIncomplete
          Status: Enabled
          AbortIncompleteMultipartUpload:
            DaysAfterInitiation: 7
    Tags:
      - Key: Name
        Value: !Sub "${AWS::StackName}-app-bucket"

AppBucketPolicy:
  Type: AWS::S3::BucketPolicy
  Properties:
    Bucket: !Ref AppBucket
    PolicyDocument:
      Version: "2012-10-17"

```

```
Statement:
  - Sid: DenyInsecureTransport
    Effect: Deny
    Principal: "*"
    Action: "s3:*"
    Resource:
      - !GetAtt AppBucket.Arn
      - !Sub "${AppBucket.Arn}/*"
    Condition:
      Bool:
        aws:SecureTransport: "false"
  - Sid: AllowEC2Access
    Effect: Allow
    Principal:
      AWS: !GetAtt EC2Role.Arn
    Action:
      - "s3:GetObject"
      - "s3:PutObject"
      - "s3:ListBucket"
    Resource:
      - !GetAtt AppBucket.Arn
      - !Sub "${AppBucket.Arn}/*"
```

2. Nested Stacks

Nested Stacks

ARQUITETURA

Nested Stacks permitem modularizar templates grandes em componentes reutilizáveis. O template pai orquestra os filhos.

```

# template-pai.yaml (root stack)
AWSTemplateFormatVersion: "2010-09-09"
Description: "Root Stack – orchestra nested stacks"

Resources:

  NetworkStack:
    Type: AWS::CloudFormation::Stack
    Properties:
      TemplateURL: https://s3.amazonaws.com/my-templates/network.yaml
      Parameters:
        VpcCidr: "10.0.0.0/16"
        Environment: !Ref Environment
      Tags:
        - Key: Component
          Value: Network

  ComputeStack:
    Type: AWS::CloudFormation::Stack
    DependsOn: NetworkStack
    Properties:
      TemplateURL: https://s3.amazonaws.com/my-templates/compute.yaml
      Parameters:
        VpcId: !GetAtt NetworkStack.Outputs.VpcId
        SubnetId: !GetAtt NetworkStack.Outputs.PublicSubnet1Id
        InstanceType: !Ref InstanceType
      Tags:
        - Key: Component
          Value: Compute

  DatabaseStack:
    Type: AWS::CloudFormation::Stack
    DependsOn: NetworkStack
    Properties:
      TemplateURL: https://s3.amazonaws.com/my-templates/database.yaml
      Parameters:
        VpcId: !GetAtt NetworkStack.Outputs.VpcId
        PrivateSubnets: !GetAtt NetworkStack.Outputs.PrivateSubnetIds
        DBPassword: !Ref DBPassword

Outputs:

  WebServerURL:

```

```
Value: !GetAtt ComputeStack.Outputs.WebServerURL
DatabaseEndpoint:
Value: !GetAtt DatabaseStack.Outputs.DatabaseEndpoint
```

3. Cross-Stack References (Export/Import)

Export OUTPUT

Outputs com Export tornam valores disponíveis para outros stacks na mesma região/conta.

```
# Stack A: network-stack
Outputs:
  VpcId:
    Description: "VPC ID"
    Value: !Ref VPC
    Export:
      Name: !Sub "${AWS::StackName}-VpcId"

  PublicSubnet1Id:
    Description: "Public Subnet 1"
    Value: !Ref PublicSubnet1
    Export:
      Name: !Sub "${AWS::StackName}-PublicSubnet1"

  PrivateSubnetIds:
    Description: "Private Subnets"
    Value: !Join
      - ","
      - - !Ref PrivateSubnet1
        - !Ref PrivateSubnet2
    Export:
      Name: !Sub "${AWS::StackName}-PrivateSubnets"
```

ImportValue [REFERÊNCIA](#)

`Fn::ImportValue` consome exports de outros stacks, criando dependências entre stacks.

```
# Stack B: compute-stack
Resources:
  WebServer:
    Type: AWS::EC2::Instance
    Properties:
      SubnetId:
        Fn::ImportValue: network-stack-PublicSubnet1

  AppSG:
    Type: AWS::EC2::SecurityGroup
    Properties:
      VpcId:
        Fn::ImportValue: network-stack-VpcId

# Listar exports disponíveis via CLI
# aws cloudformation list-exports \
#   --query "Exports[].{Name:Name,Value:Value}"

# Listar stacks que importam um export
# aws cloudformation list-imports \
#   --export-name network-stack-VpcId
```

4. Best Practices

Boas Práticas **PRODUÇÃO**

Práticas essenciais para gerenciar stacks CloudFormation em produção com segurança.


```
# 1. SEMPRE use Change Sets antes de atualizar stacks em produção
aws cloudformation create-change-set \
  --stack-name prod-stack \
  --change-set-name release-v2.1 \
  --template-body file:///template.yaml

# 2. Drift Detection – detectar mudanças manuais
aws cloudformation detect-stack-drift \
  --stack-name minha-stack

aws cloudformation describe-stack-drift-detection-status \
  --stack-drift-detection-id <detection-id>

aws cloudformation describe-stack-resource-drifts \
  --stack-name minha-stack \
  --stack-resource-drift-status-filters MODIFIED DELETED

# 3. Stack Policy – proteger recursos críticos contra updates
aws cloudformation set-stack-policy \
  --stack-name prod-stack \
  --stack-policy-body '{
    "Statement": [{
      "Effect": "Deny",
      "Action": "Update:Replace",
      "Principal": "*",
      "Resource": "LogicalResourceId/Database"
    }, {
      "Effect": "Allow",
      "Action": "Update:*",
      "Principal": "*",
      "Resource": "*"
    }]
  }'

# 4. DeletionPolicy e UpdateReplacePolicy
#   - Retain: mantém recurso ao deletar stack
#   - Snapshot: cria snapshot antes de deletar (RDS, EBS)
#   - Delete: remove recurso (padrão)

# 5. Termination Protection
aws cloudformation update-termination-protection \
```

```
--enable-termination-protection \  
--stack-name prod-stack  
  
# 6. Tagging consistente em todos os recursos  
# 7. Usar SSM Parameter Store para valores dinâmicos (AMIs, etc.)  
# 8. Validar templates com cfn-lint antes do deploy  
# pip install cfn-lint  
# cfn-lint template.yaml
```

Labs Práticos

4 Labs para Consolidar o Aprendizado

Aplique cada módulo em labs progressivos que constroem uma arquitetura completa na AWS.

Lab 1: Template Básico — VPC Simples

lab01-vpc.yaml

LAB 01

```

AWSTemplateFormatVersion: "2010-09-09"

Description: "Lab 01 – VPC básica com subnet pública"

Parameters:
  ProjectName:
    Type: String
    Default: cloudtrilhas-lab01

Resources:
  VPC:
    Type: AWS::EC2::VPC
    Properties:
      CidrBlock: "10.0.0.0/16"
      EnableDnsSupport: true
      EnableDnsHostnames: true
      Tags:
        - Key: Name
          Value: !Sub "${ProjectName}-vpc"

  PublicSubnet:
    Type: AWS::EC2::Subnet
    Properties:
      VpcId: !Ref VPC
      CidrBlock: "10.0.1.0/24"
      AvailabilityZone: !Select [0, !GetAZs ""]
      MapPublicIpOnLaunch: true
      Tags:
        - Key: Name
          Value: !Sub "${ProjectName}-public"

  IGW:
    Type: AWS::EC2::InternetGateway

  AttachIGW:
    Type: AWS::EC2::VPCGatewayAttachment
    Properties:
      VpcId: !Ref VPC
      InternetGatewayId: !Ref IGW

Outputs:
  VpcId:

```

```
Value: !Ref VPC
Export:
  Name: !Sub "${ProjectName}-VpcId"
SubnetId:
  Value: !Ref PublicSubnet
Export:
  Name: !Sub "${ProjectName}-SubnetId"

# Deploy:
# aws cloudformation create-stack --stack-name lab01 \
#   --template-body file:///lab01-vpc.yaml
```

Lab 2: VPC + EC2 com Apache

lab02-ec2.yaml

LAB 02

```

AWSTemplateFormatVersion: "2010-09-09"

Description: "Lab 02 – EC2 com UserData e CreationPolicy"

Parameters:

  KeyPair:
    Type: AWS::EC2::KeyPair::KeyName

  VpcId:
    Type: AWS::EC2::VPC::Id

  SubnetId:
    Type: AWS::EC2::Subnet::Id

Resources:

  WebSG:
    Type: AWS::EC2::SecurityGroup
    Properties:
      GroupDescription: "HTTP + SSH"
      VpcId: !Ref VpcId
      SecurityGroupIngress:
        - IpProtocol: tcp
          FromPort: 80
          ToPort: 80
          CidrIp: "0.0.0.0/0"
        - IpProtocol: tcp
          FromPort: 22
          ToPort: 22
          CidrIp: "0.0.0.0/0"

  WebServer:
    Type: AWS::EC2::Instance
    CreationPolicy:
      ResourceSignal:
        Timeout: PT5M
    Properties:
      ImageId: !Sub "{{resolve:ssm:/aws/service/ami-amazon-linux-latest/amzn2-ami-hvm-
      InstanceType: t3.micro
      KeyName: !Ref KeyPair
      SubnetId: !Ref SubnetId
      SecurityGroupIds:
        - !Ref WebSG
      UserData:
        Fn::Base64: !Sub |

```

```
#!/bin/bash -xe
yum update -y
yum install -y httpd
systemctl start httpd
systemctl enable httpd
echo "<h1>Lab 02 - ${AWS::StackName}</h1>" > /var/www/html/index.html
/opt/aws/bin/cfn-signal -e $? --stack ${AWS::StackName} --resource WebServer
```

Outputs:

URL:

Value: !Sub "http://\${WebServer.PublicDnsName}"

Lab 3: VPC + EC2 + RDS MySQL

lab03-rds.yaml

LAB 03

```
AWSTemplateFormatVersion: "2010-09-09"

Description: "Lab 03 – Adicionando RDS com Conditions"

Parameters:
  Environment:
    Type: String
    Default: dev
    AllowedValues: [dev, prod]
  DBPassword:
    Type: String
    NoEcho: true
    MinLength: 8

Conditions:
  IsProd: !Equals [!Ref Environment, prod]

Mappings:
  DBConfig:
    dev:
      Class: db.t3.micro
      Storage: 20
      MultiAZ: "false"
    prod:
      Class: db.r5.large
      Storage: 100
      MultiAZ: "true"

Resources:
  DBSubnetGroup:
    Type: AWS::RDS::DBSubnetGroup
    Properties:
      DBSubnetGroupDescription: "Subnets privadas para RDS"
      SubnetIds:
        - !Ref PrivateSubnet1
        - !Ref PrivateSubnet2

  DBSG:
    Type: AWS::EC2::SecurityGroup
    Properties:
      GroupDescription: "RDS MySQL"
      VpcId: !Ref VPC
```

```

SecurityGroupIngress:
  - IpProtocol: tcp
    FromPort: 3306
    ToPort: 3306
    SourceSecurityGroupId: !Ref WebSG

MySQLDB:
  Type: AWS::RDS::DBInstance
  DeletionPolicy: Snapshot
  Properties:
    Engine: mysql
    EngineVersion: "8.0"
    DBInstanceClass: !FindInMap [DBConfig, !Ref Environment, Class]
    AllocatedStorage: !FindInMap [DBConfig, !Ref Environment, Storage]
    MultiAZ: !FindInMap [DBConfig, !Ref Environment, MultiAZ]
    MasterUsername: admin
    MasterUserPassword: !Ref DBPassword
    DBSubnetGroupName: !Ref DBSubnetGroup
    VPCSecurityGroups:
      - !Ref DBSG
    StorageEncrypted: true
    DeletionProtection: !If [IsProd, true, false]

Outputs:
  DBEndpoint:
    Value: !GetAtt MySQLDB.Endpoint.Address
    Export:
      Name: !Sub "${AWS::StackName}-DBEndpoint"

# Deploy:
# aws cloudformation create-stack --stack-name lab03 \
#   --template-body file://lab03-rds.yaml \
#   --parameters ParameterKey=DBPassword,ParameterValue=MinhaSenha@123

```

Lab 4: Arquitetura Completa — VPC + EC2 + RDS + S3

lab04-full.yaml

LAB 04

```

AWSTemplateFormatVersion: "2010-09-09"

Description: "Lab 04 – Arquitetura completa com Nested Stacks"

Parameters:
  Environment:
    Type: String
    Default: dev
    AllowedValues: [dev, staging, prod]
  TemplateBucket:
    Type: String
    Description: "Bucket S3 com os templates filhos"

Resources:
  NetworkStack:
    Type: AWS::CloudFormation::Stack
    Properties:
      TemplateURL: !Sub "https://${TemplateBucket}.s3.amazonaws.com/network.yaml"
      Parameters:
        Environment: !Ref Environment
        VpcCidr: "10.0.0.0/16"

  ComputeStack:
    Type: AWS::CloudFormation::Stack
    DependsOn: NetworkStack
    Properties:
      TemplateURL: !Sub "https://${TemplateBucket}.s3.amazonaws.com/compute.yaml"
      Parameters:
        VpcId: !GetAtt NetworkStack.Outputs.VpcId
        PublicSubnetId: !GetAtt NetworkStack.Outputs.PublicSubnet1Id
        Environment: !Ref Environment

  DatabaseStack:
    Type: AWS::CloudFormation::Stack
    DependsOn: NetworkStack
    Properties:
      TemplateURL: !Sub "https://${TemplateBucket}.s3.amazonaws.com/database.yaml"
      Parameters:
        VpcId: !GetAtt NetworkStack.Outputs.VpcId
        PrivateSubnets: !GetAtt NetworkStack.Outputs.PrivateSubnetIds
        WebServerSGId: !GetAtt ComputeStack.Outputs.WebServerSGId
        Environment: !Ref Environment

```

```

StorageStack:
  Type: AWS::CloudFormation::Stack
  Properties:
    TemplateURL: !Sub "https://${TemplateBucket}.s3.amazonaws.com/storage.yaml"
    Parameters:
      Environment: !Ref Environment
      EC2RoleArn: !GetAtt ComputeStack.Outputs.EC2RoleArn

Outputs:
  WebURL:
    Description: "URL da aplicação"
    Value: !GetAtt ComputeStack.Outputs.WebServerURL
  DBEndpoint:
    Description: "Endpoint do banco"
    Value: !GetAtt DatabaseStack.Outputs.DBEndpoint
  S3Bucket:
    Description: "Bucket da aplicação"
    Value: !GetAtt StorageStack.Outputs.BucketName

# Deploy:
# 1. Upload templates para S3
# aws s3 cp ./templates/ s3://my-cfn-templates/ --recursive
#
# 2. Deploy root stack
# aws cloudformation create-stack --stack-name lab04-full \
#   --template-body file://lab04-full.yaml \
#   --parameters ParameterKey=TemplateBucket,ParameterValue=my-cfn-templates \
#   --capabilities CAPABILITY_NAMED_IAM
#
# 3. Verificar
# aws cloudformation describe-stacks --stack-name lab04-full

```